

Face Detection & Recognition I

Unter **Face Detection** versteht man im Wesentlichen die Fähigkeit, die Position eines Gesichtes in einem beliebigen Bild zu erkennen. Um diese Aufgabe zu bewältigen, kann man entweder selbst ein entsprechendes System¹ entwickeln, oder auf bestehende Ressourcen zurückgreifen. Nachfolgender Ansatz basiert auf der Python-Bibliothek *face_recognition*², die wiederum auf *dlib*³ aufbaut. Eine überaus brauchbare Alternative wäre die Bibliothek *OpenCV*⁴, die wir im Rahmen des Unterrichts ebenfalls noch benötigen werden.

Warum *face_recognition*? Weil's funktioniert und einfach zu verwenden ist. Die Installation (inkl. der notwendigen Abhängigkeiten) ist auf der Github-Site ausführlich dokumentiert. Zusammengefasst braucht es folgende Schritte:

1. Installation von *cmake*⁵. **Wichtig:** Bei der Systemvariable `Path` gilt es einen Eintrag zu setzen, der in das bin-Verzeichnis der Installation zeigt (z.B. `C:\Program Files\CMake\bin`).
2. Installation von *dlib*: `pip install dlib`
3. Installation von *face_recognition*: `pip install face_recognition`

Anwendungsbeispiel Face Detection⁶:

```
from PIL import Image
import face_recognition

# Load the jpg file into a numpy array
image = face_recognition.load_image_file("data/biden.jpg")

# Find all the faces in the image using the default HOG-based model.
# This method is fairly accurate, but not as accurate as the CNN model and
# not GPU accelerated.
# See also: find_faces_in_picture_cnn.py
face_locations = face_recognition.face_locations(image)

print("I found {} face(s) in this photograph.".format(len(face_locations)))

for face_location in face_locations:
```

¹ Mittlerweile sollte eine grobe Vorstellung bzgl. des Aufwandes vorhanden sein!

² https://github.com/ageitgey/face_recognition bzw. <https://face-recognition.readthedocs.io/en/latest/index.html>

³ <http://dlib.net/>

⁴ <https://opencv.org/>

⁵ <https://cmake.org/download/>

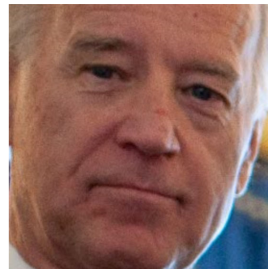
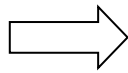
⁶ https://github.com/ageitgey/face_recognition/blob/master/examples/find_faces_in_picture.py

```
# Print the location of each face in this image
top, right, bottom, left = face_location
print("A face is located at pixel location Top: {}, Left: {}, Bottom:
{}, Right: {}".format(top, left, bottom, right))

# You can access the actual face itself like this:
face_image = image[top:bottom, left:right]
pil_image = Image.fromarray(face_image)
pil_image.show()
```

Output:

I found 1 face(s) in this photograph.
A face is located at pixel location Top: 241, Left: 419, Bottom: 562,
Right: 740



 21.1 Installieren Sie sämtliche Ressourcen und testen Sie das angeführte Code-Beispiel.
Hinweis: Das Bild stammt aus dem Github-Repo (`examples/biden.jpg`)

Gegeben ist folgendes Bild⁷:

⁷ <https://www.pexels.com/de-de/foto/multikulturelle-menschen-3184419/>



✎ 21.2 Gesucht ist folgender Output:

Auf dem Bild sind 5 zu sehen.

```
[(1617, 4099, 2002, 3713), (1060, 3029, 1446, 2643), (1274, 1188, 1660, 803), (1446, 2044, 1831, 1659), (1490, 5163, 1811, 4842)]
```

Hinweis: Das jeweilige Tupel enthält den Return-Wert der Funktion `face_locations`⁸. Entsprechende Beispiele sind der Dokumentation zu entnehmen.

Face Recognition verfolgt das Ziel zu erkennen, wer sich auf dem Bild befindet.

✎ 21.3 Setzen Sie (mithilfe der bereitgestellten Ressourcen) ein repräsentatives Beispiel um.

Just for fun

Mit dieser Bibliothek kann man auch richtig kindisch sein, wie dieses Bild zeigt (siehe Augenbrauen, Augen und Lippen):

⁸ https://face-recognition.readthedocs.io/en/latest/face_recognition.html#face_recognition.api.face_locations



 21.4 Finden Sie heraus, wie man zu diesem Ergebnis⁹ gelangt. Setzen Sie ein Merkmal bei sich selbst um (Selfie).

⁹ Ist echt nicht böse gemeint, Mr. President!